

further topics

stats403_deep_learning
spring_2025
lecture_13

13.1 deep reinforcement learning

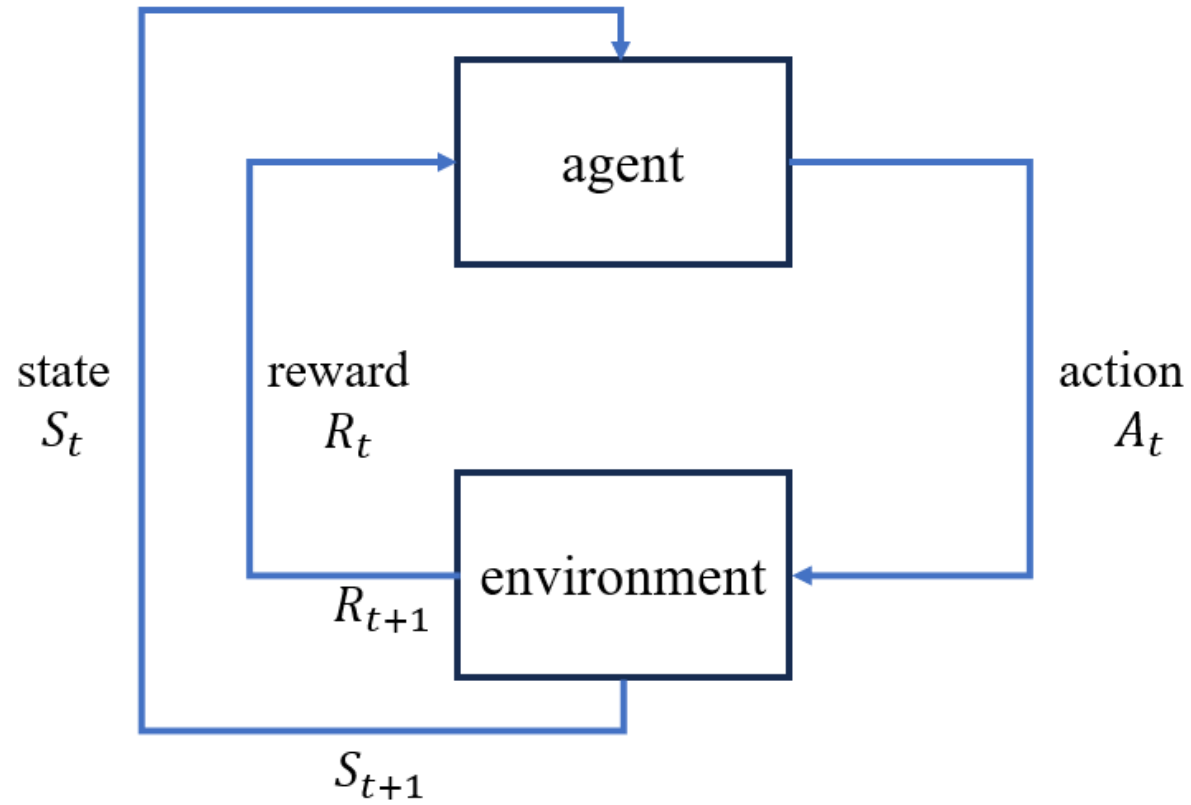
reinforcement learning

- Many real-world problems, such as game playing, robotics and autonomous driving, involve decision making under complex and dynamic environments characterized by uncertainty, incompleteness and high dimensionality.
- For instance, in game playing, the environment involves the state of the game, the move of the opponent and the potential outcomes. Supervised or unsupervised learning methods do not fit to solve these kind of problems.
- Instead, reinforcement learning (RL) involves an agent learning to make a sequence of decisions in an environment. The agent interacts with the environment through a series of actions and receives feedback in the form of rewards or punishments, depending on the actions taken.

reinforcement learning

- A typical reinforcement learning algorithm involves an agent that interacts with an environment in a series of discrete time steps.
- At each time step, the agent receives an observation of the current state of the environment and selects an action to perform based on its current policy.
- The environment then transitions to a new state, and the agent receives a scalar reward signal indicating how well it performed in the previous state.

reinforcement learning

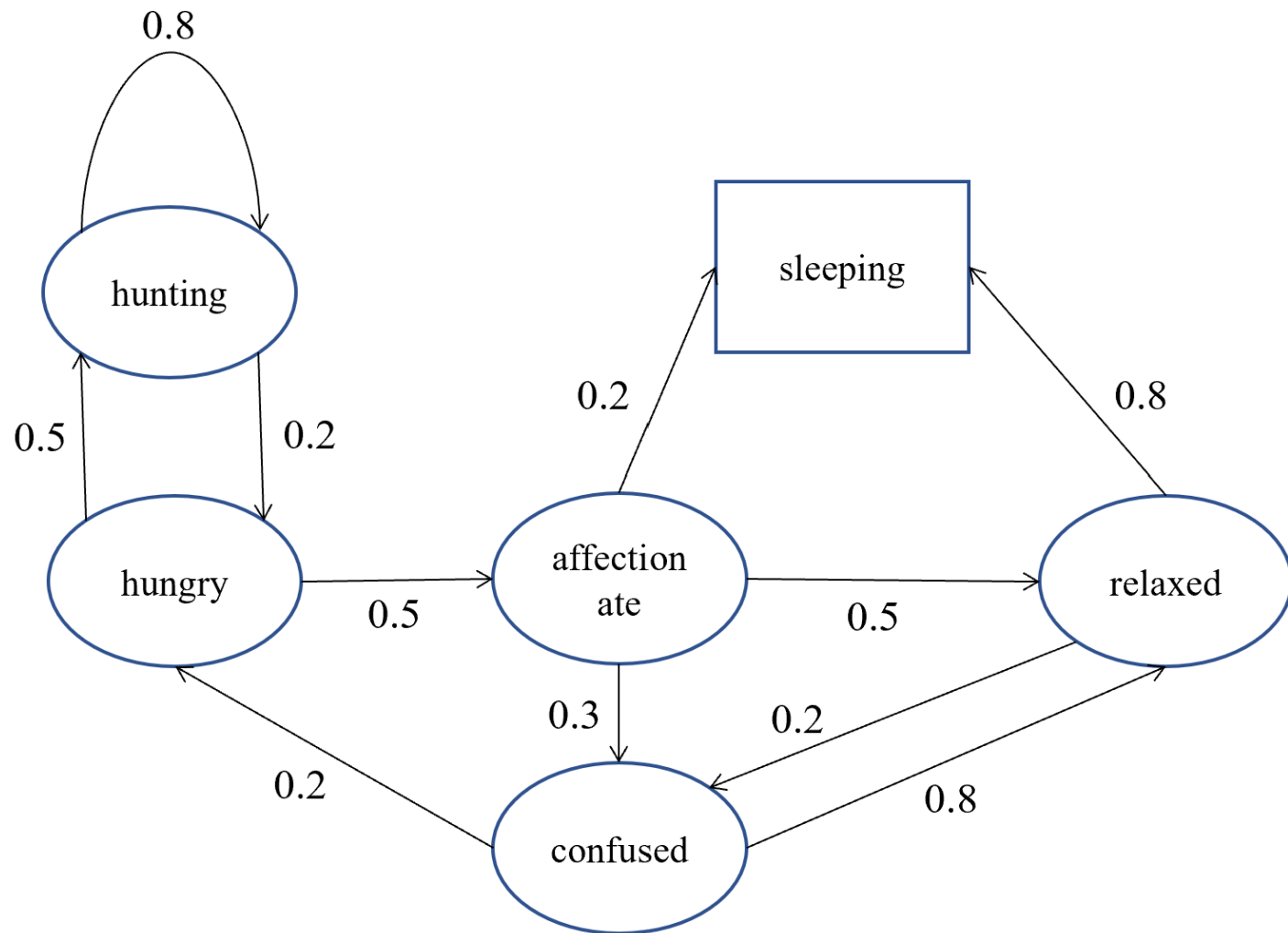


markov decision process (MDP)

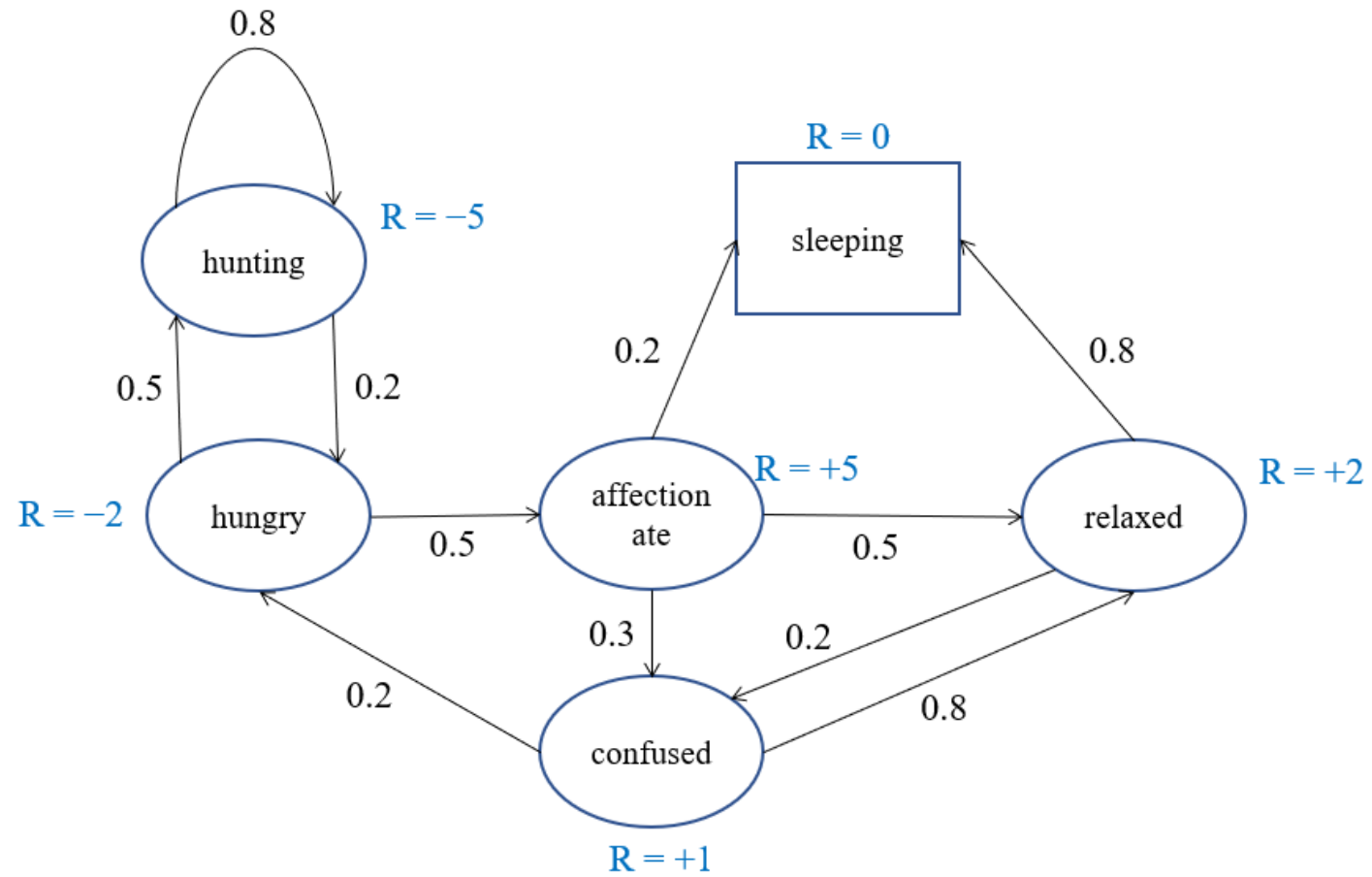
- markov decision processes formally describe an environment for reinforcement learning
- the environment is fully observable
- almost all reinforcement learning problems are MDPs

example

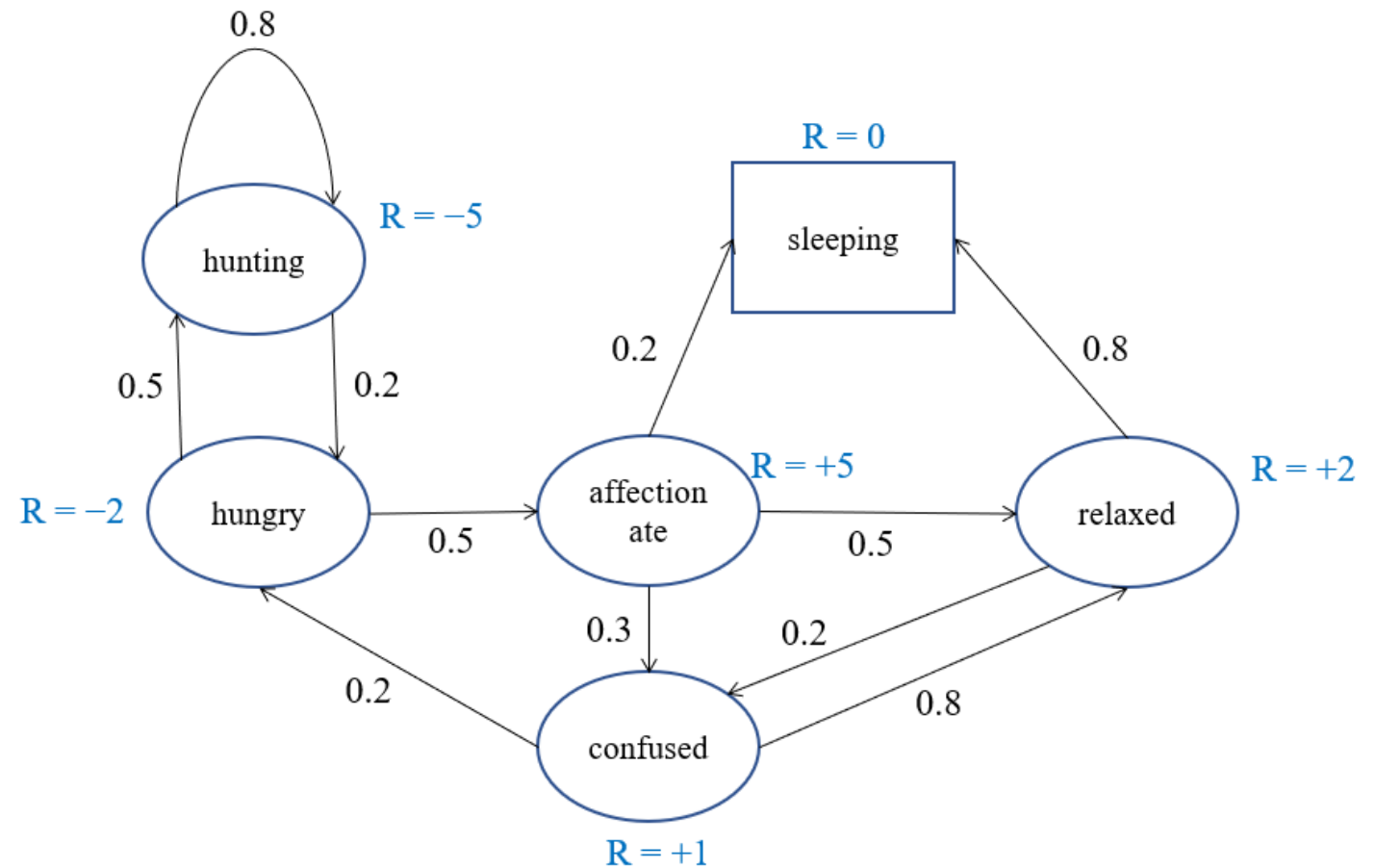
markov chain



markov reward process (MRP)

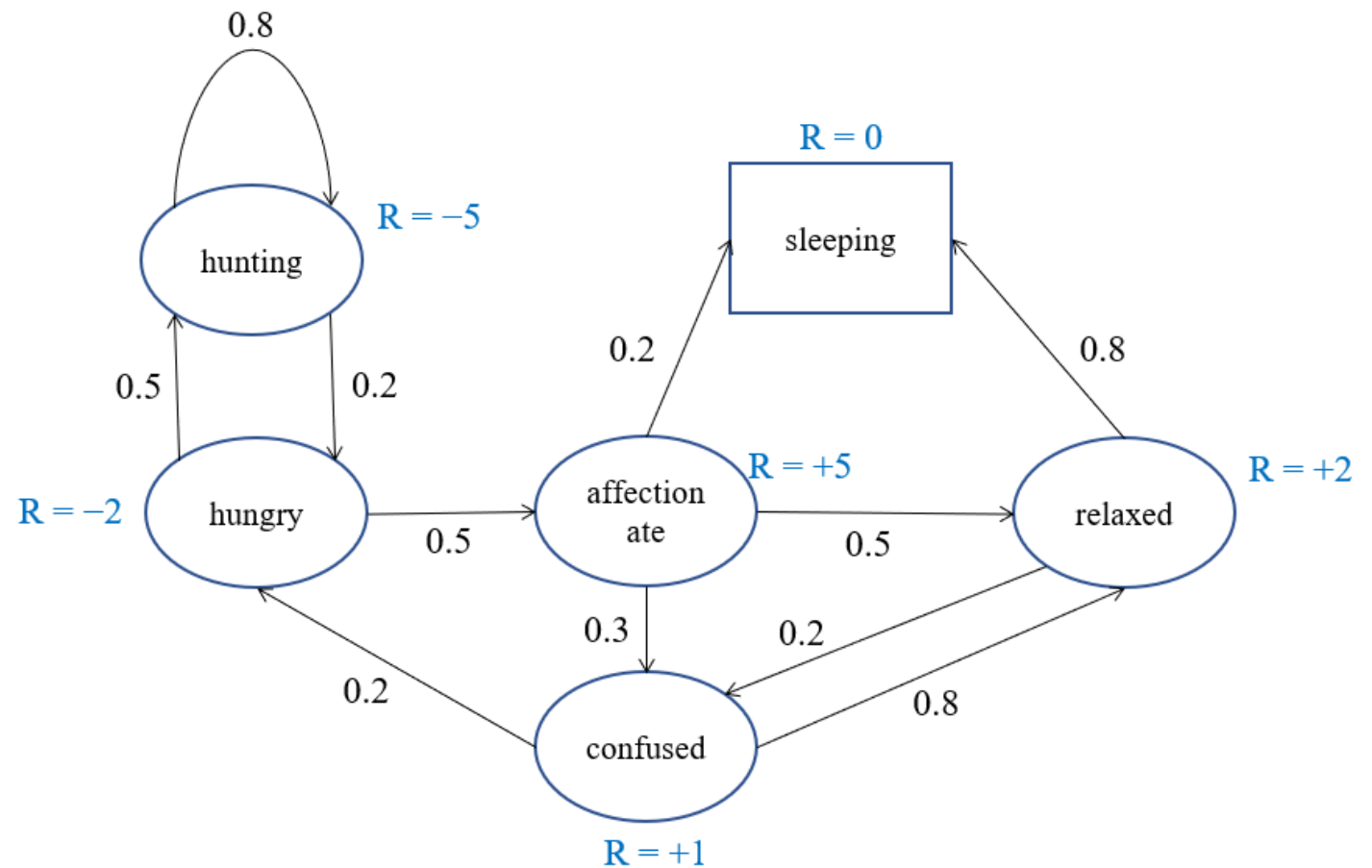


Let's calculate the "value" of:
hungry -> hunting -> hungry -> affectionate -> sleeping



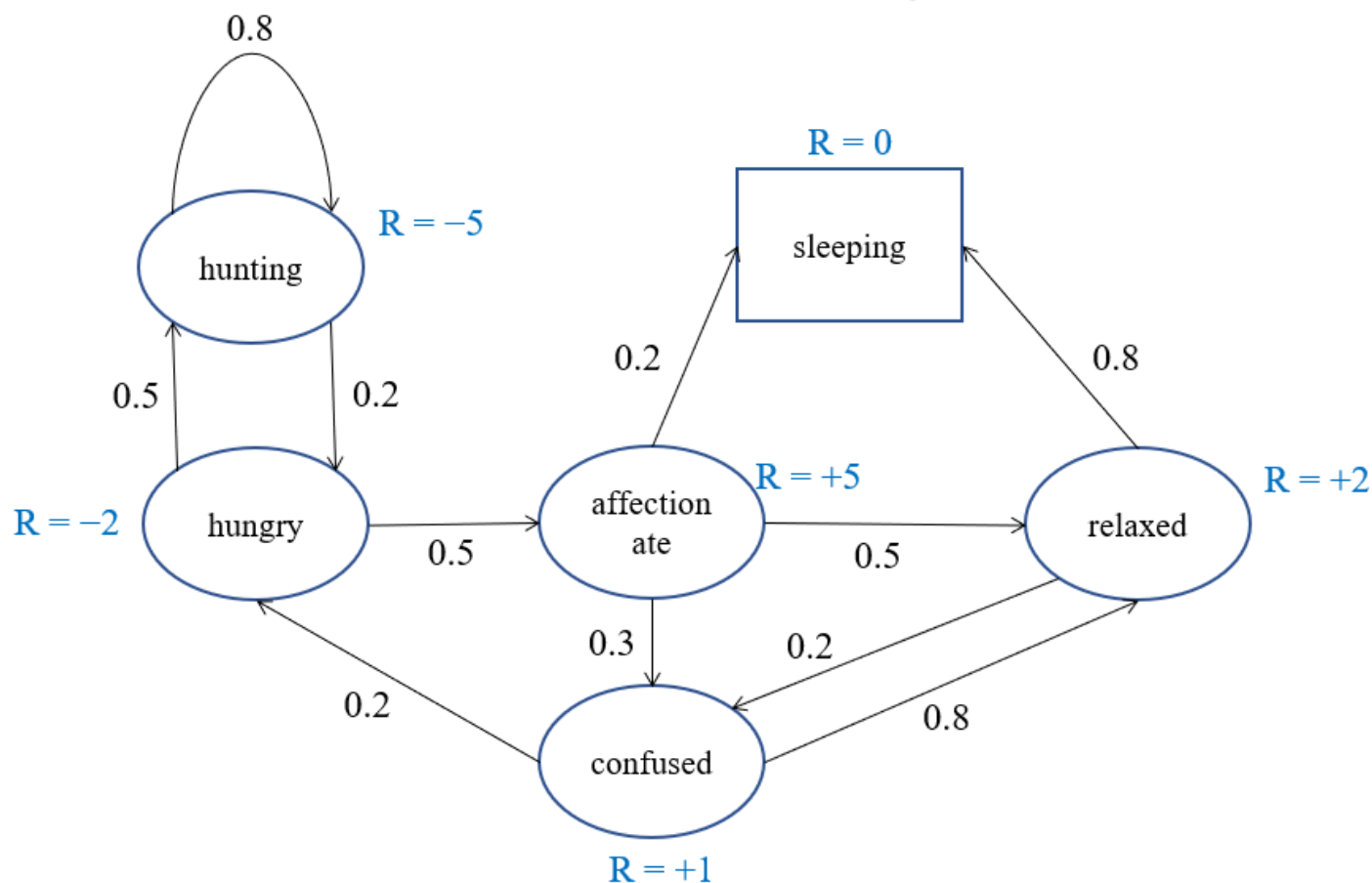
Let's calculate the "value" of:
hungry -> hunting -> hungry -> affectionate -> sleeping

Rule: discount the future rewards by 0.9



value as a function of states

We can also assign values to each state by setting a system of equations following the one-step process. Since sleeping is a terminal state with zero reward, its value has to be zero. Denote the values of the other states, hunting, hungry, affectionate, relaxed, confused, as $x_1, \dots, x_5$, respectively.



value as a function of states

We can also assign values to each state by setting a system of equations following the one-step process. Since sleeping is a terminal state with zero reward, its value has to be zero. Denote the values of the other states, hunting, hungry, affectionate, relaxed, confused, as $x_1, \dots, x_5$, respectively. We have

$$x_1 = -5 + 0.8\gamma x_1 + 0.2\gamma x_2$$

$$x_2 = -2 + 0.5\gamma x_1 + 0.5\gamma x_3$$

$$x_3 = 5 + 0.3\gamma x_5 + 0.5\gamma x_4$$

$$x_4 = 2 + 0.2\gamma x_5$$

$$x_5 = 1 + 0.2\gamma x_2 + 0.8\gamma x_4$$

For $\gamma = 0.9$, we have approximately $x_1 = -24.4$, $x_2 = -10.2$, $x_3 = 6.1$, $x_4 = 2.1$ and $x_5 = 0.7$. This is a very simple example of the well-known Bellman Equation.

reinforcement learning: we can take actions!

- **exploiting**: take the “best” action according to your current estimation
 - e.g., go to your favorite restaurant, apply to an established program with long history
- **exploring**: take a nongreedy action, but may gather more information, thus having better reward in the long run
 - e.g., try a new restaurant, apply to a new ECE program

reinforcement learning

- Value (Q) is a function of State (S) and Action (A)

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Let's try to understand the idea through the exciting example of AlphaGo!

deep Q-learning

- in deep Q-learning, a neural network $Q(s, a; \boldsymbol{\theta})$ is used to estimate the Q values given the state-action pairs (s, a)
- e.g.,

$$L_i(\boldsymbol{\theta}_i) = \mathbb{E} \left[\left(r + \gamma \max'_a Q(s', a'; \boldsymbol{\theta}_{i-1}) - Q(s, a; \boldsymbol{\theta}_i) \right)^2 \right]$$

$$\nabla L_i(\boldsymbol{\theta}_i) = \mathbb{E} \left[\left(r + \gamma \max'_a Q(s', a'; \boldsymbol{\theta}_{i-1}) - Q(s, a; \boldsymbol{\theta}_i) \right) \nabla Q(s, a; \boldsymbol{\theta}_i) \right]$$

Published: 27 January 2016

Mastering the game of Go with deep neural networks and tree search

David Silver , Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel & Demis Hassabis 

ARTICLE

doi:10.1038/nature24270

Mastering the game of Go without human knowledge

David Silver^{1*}, Julian Schrittwieser^{1*}, Karen Simonyan^{1*}, Ioannis Antonoglou¹, Aja Huang¹, Arthur Guez¹, Thomas Hubert¹, Lucas Baker¹, Matthew Lai¹, Adrian Bolton¹, Yutian Chen¹, Timothy Lillicrap¹, Fan Hui¹, Laurent Sifre¹, George van den Driessche¹, Thore Graepel¹ & Demis Hassabis¹

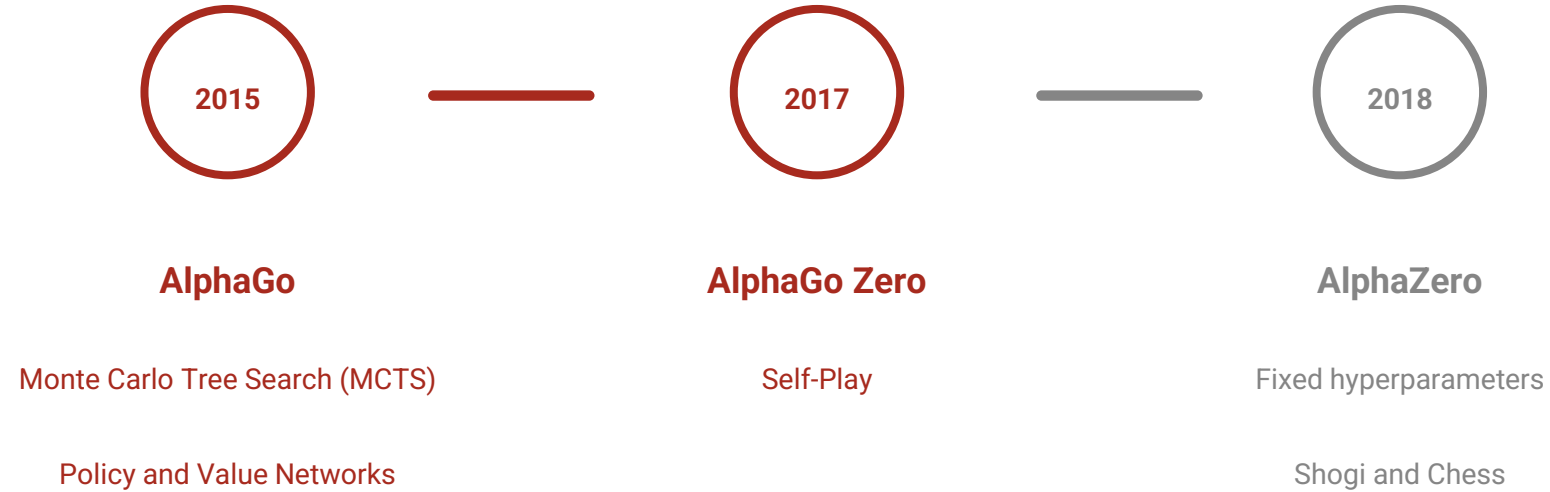
Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm

David Silver,^{1*} Thomas Hubert,^{1*} Julian Schrittwieser,^{1*} Ioannis Antonoglou,¹ Matthew Lai,¹ Arthur Guez,¹ Marc Lanctot,¹ Laurent Sifre,¹ Dhharshan Kumaran,¹ Thore Graepel,¹ Timothy Lillicrap,¹ Karen Simonyan,¹ Demis Hassabis¹

¹DeepMind, 6 Pancras Square, London N1C 4AG.

*These authors contributed equally to this work.

AlphaGo to AlphaZero



We can't afford to train

For Alphazero

To train the network, need 4 TPU

To generate the self-play, need 5,000 TPU

Gomoku?

https://github.com/junxiaosong/AlphaZero_Gomoku



AlphaGo Zero: reinforcement learning via selfplay

- a policy and value network (in AlphaGo, separate networks)
- MCTS guided by the network
- self-play via MCTS to generate training data (in AlphaGo, training data is from human)
- update network using the generated data

one network for both policy and value

use deep neural network

$$(\mathbf{p}, v) = f_{\theta}(s)$$

- input: state of the board
- output:
 - move probability: $p_a = \Pr(a|s)$
 - value (probability of the current player winning from position s)

network structure

input:

State: 19-by-19-by-17 binary

- 19-by-19 is the board size
- $17 = 8$ (black history) + 8 (white history) + 1 (which player to play)

This is due to special rules for competition

layers: convolution, residual (batch normalization, ReLU)

39 or 79 layers

network structure

output:

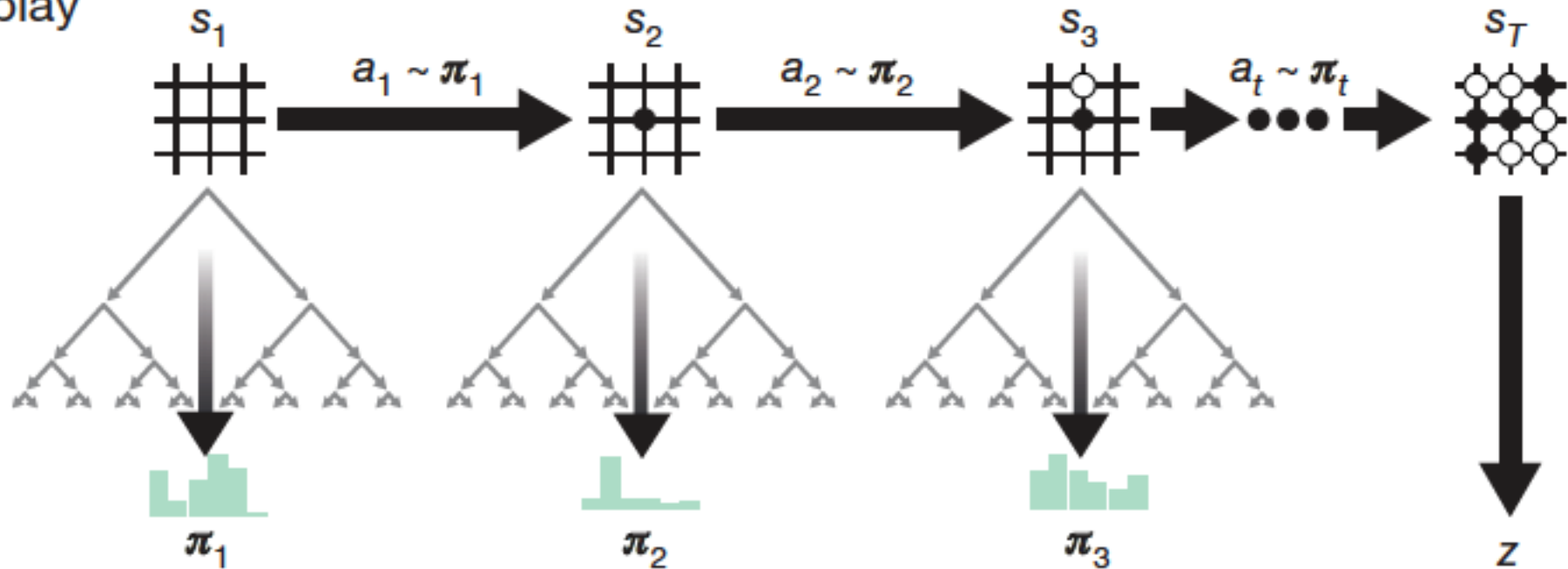
policy: a vector of size $19 \times 19 + 1 = 362$

(player may choose to pass)

value: a scalar in $[-1, 1]$

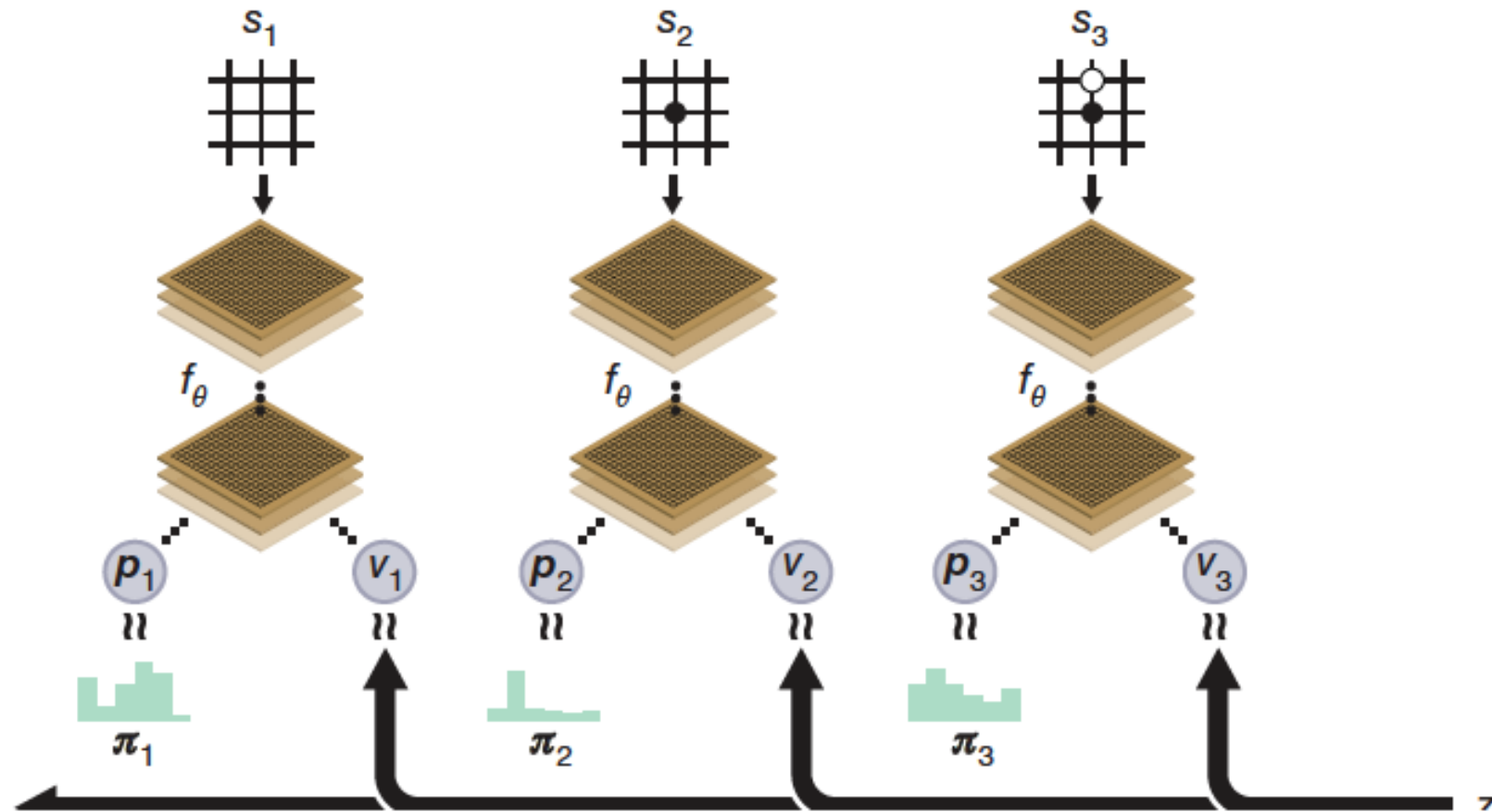
self-play reinforcement learning in AlphaGo Zero

a Self-play



self-play reinforcement learning in AlphaGo Zero

b Neural network training



optimization

$$l = (z - v)^2 - \boldsymbol{\pi}^T \log \boldsymbol{p} + c \|\boldsymbol{\theta}\|^2$$

loss function:

z : self-play result

$\boldsymbol{\pi}$: search probability

c : regularization to prevent overfitting

evaluator

after 1,000 training steps, produce a checkpoint

evaluate the checkpoint

- if better, replace the current best network for generating data (self-play)
- else, still use the previous benchmark

in Alphazero:

always use the most current one

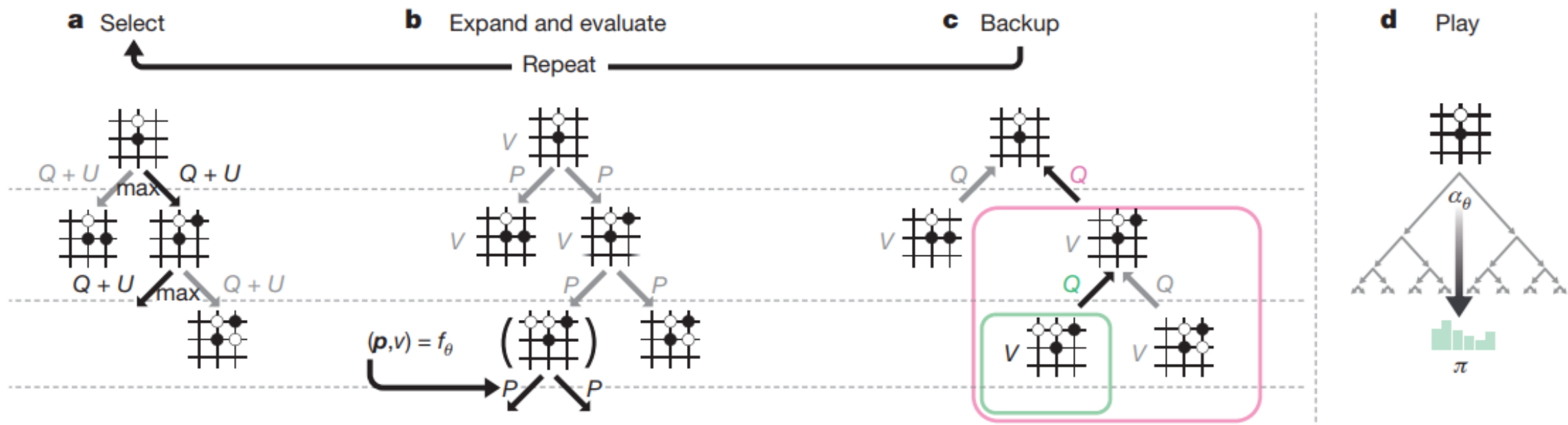
self-play

generate data:

- play 25,000 games of self-play

- using 1,600 simulations of MCTS to select each move

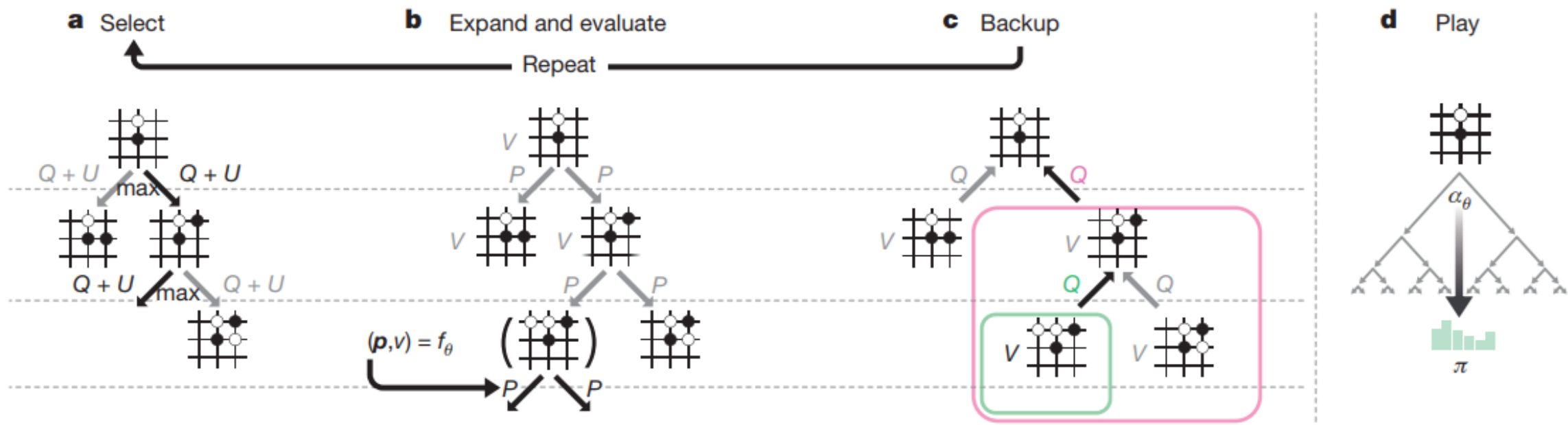
MCTS



Q : action value

U : upper confidence bound, decreases as number of visit increases

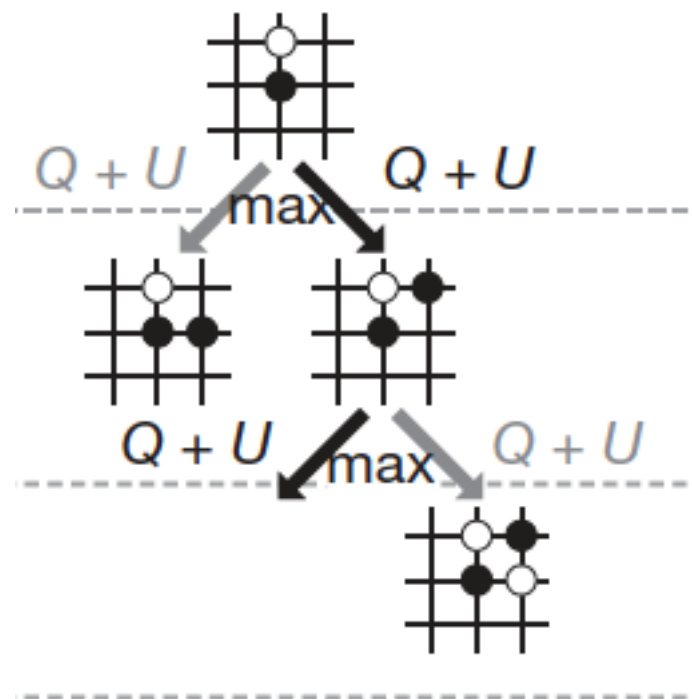
MCTS



stops when a winner is determined

MCTS

select

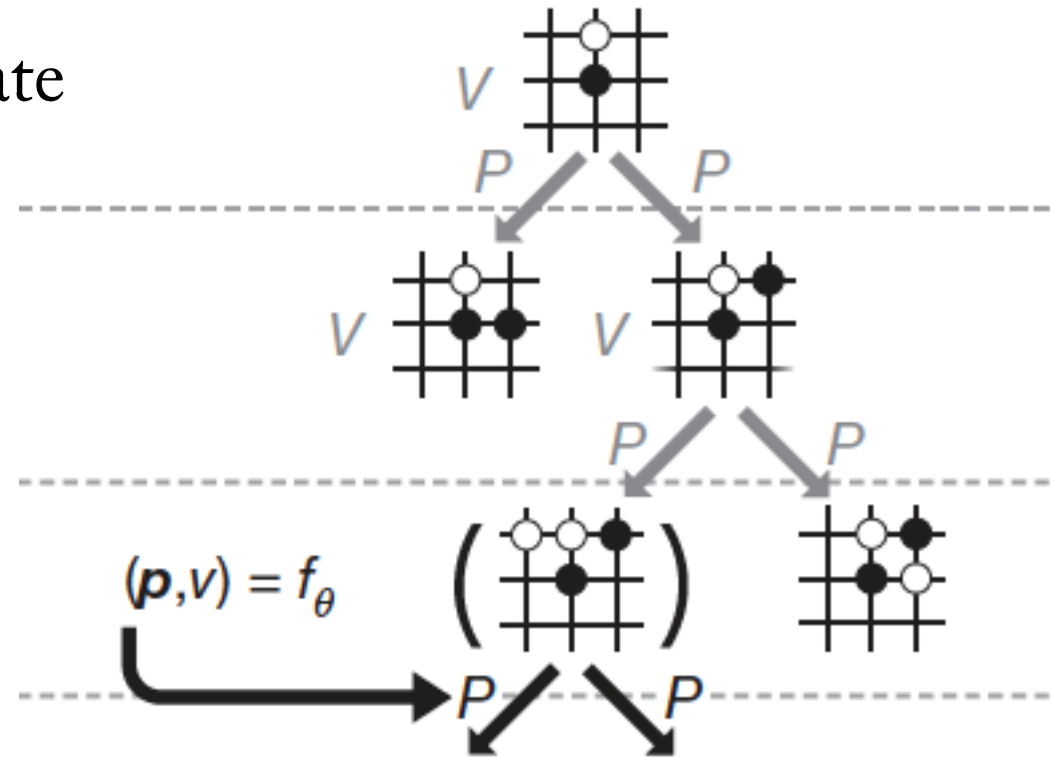


$$a_t = \underset{a}{\operatorname{argmax}}(Q(s_t, a) + U(s_t, a))$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

MCTS

expand and evaluate

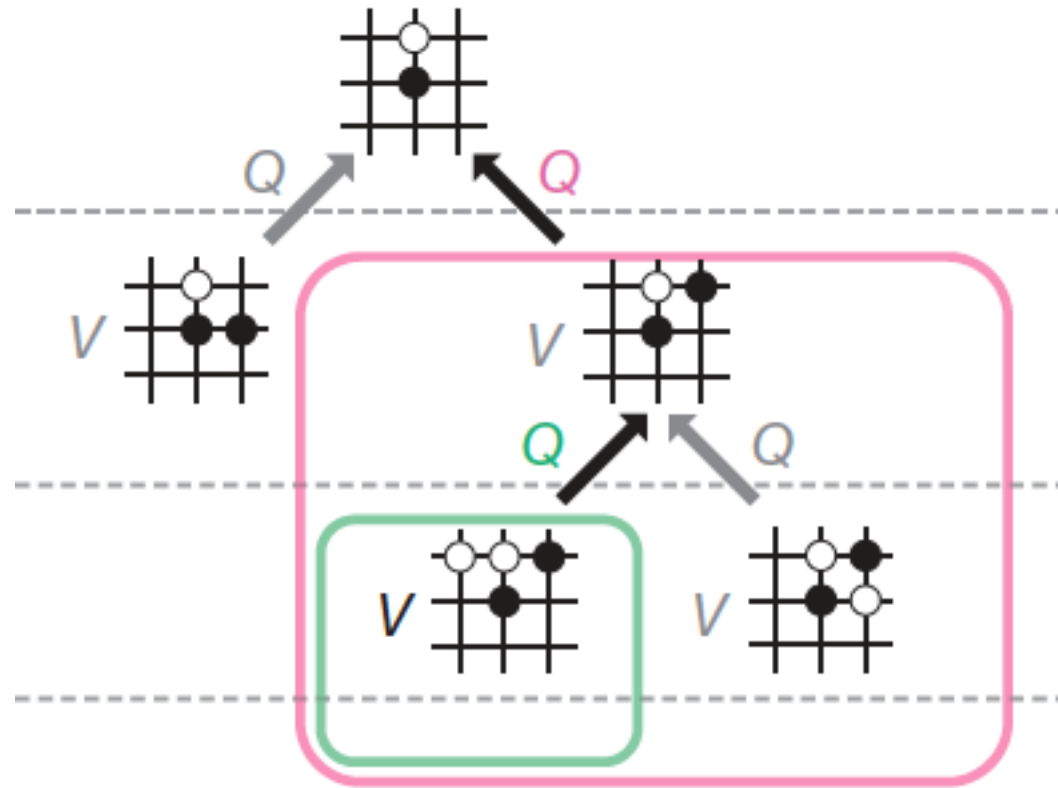


initialize for new edge

$$\{N(s_L, a) = 0, W(s_L, a) = 0, Q(s_L, a) = 0, P(s_L, a) = p_a\}$$

MCTS

backup

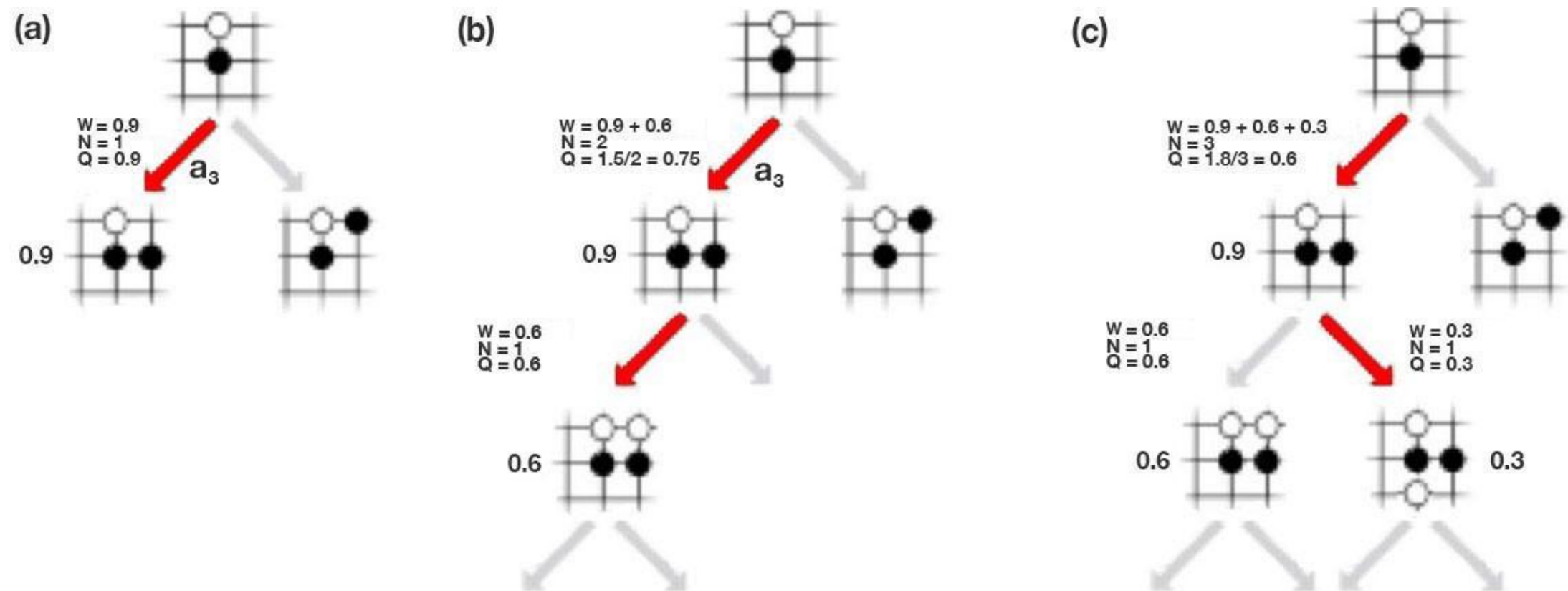


$$N(s_t, a_t) = N(s_t, a_t) + 1$$

$$W(s_t, a_t) = W(s_t, a_t) + v$$

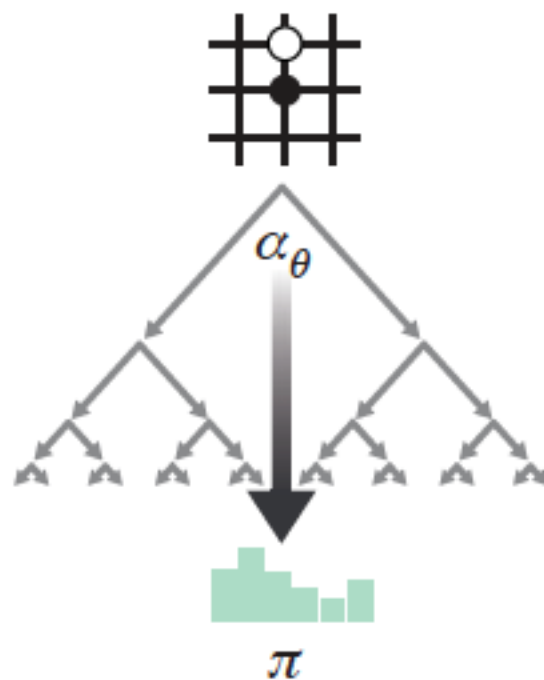
$$Q(s_t, a_t) = \frac{W(s_t, a_t)}{N(s_t, a_t)}$$

MCTS



MCTS

play



$$\pi(a|s_0) = N(s_0, a)^{1/\tau} / \sum_b N(s_0, b)^{1/\tau}$$

Alphazero

Chess and Shogi

asymmetric board

different state and policy space

e.g. policy space for chess: 8-by-8-by-73

$73 = 7 \times 8$ (step $\times$ direction) +

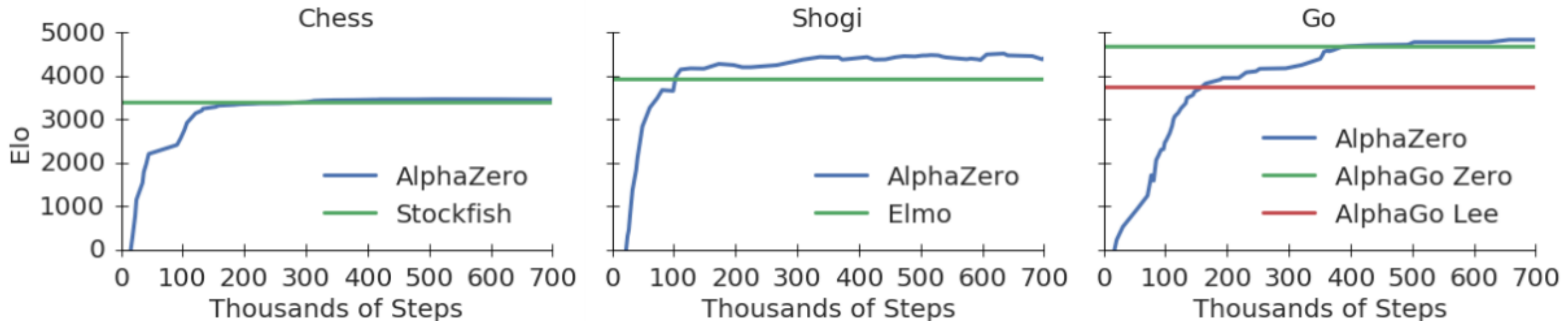
8 (knight move) +

9 (underpromotion)

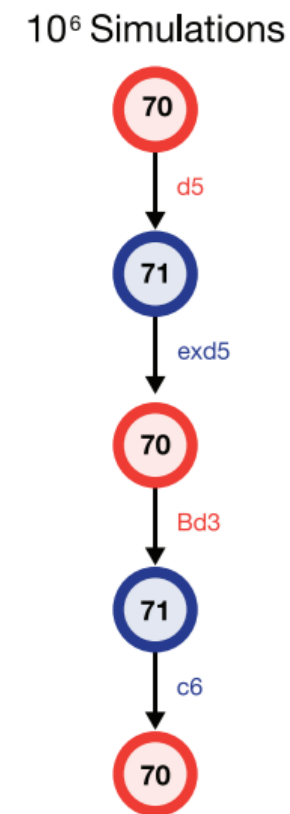
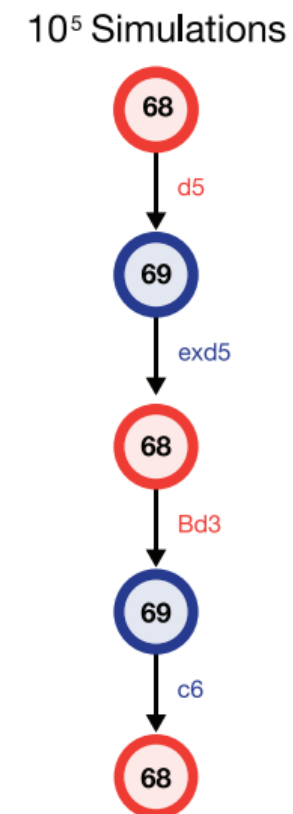
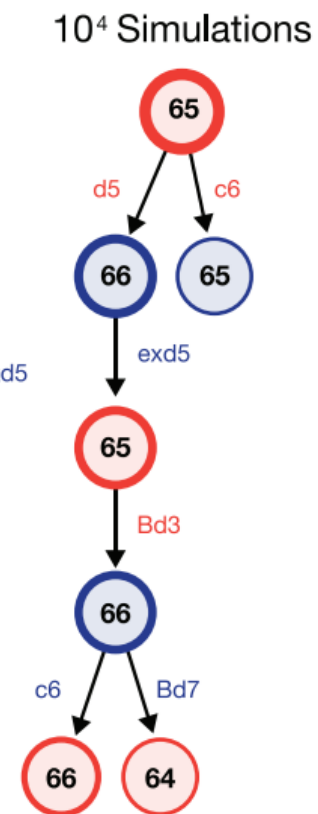
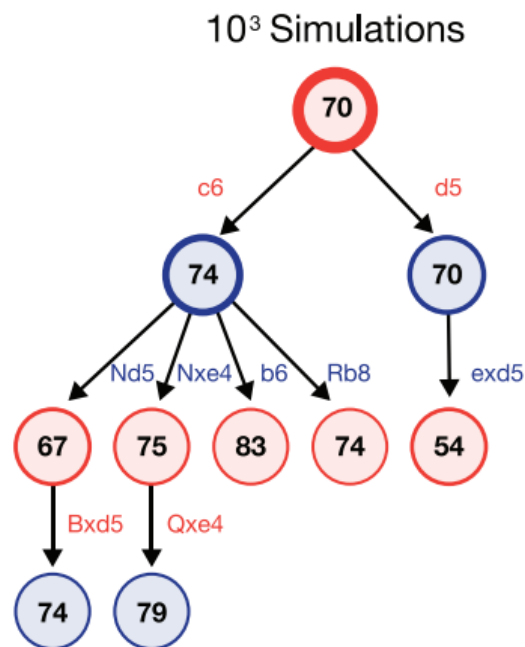
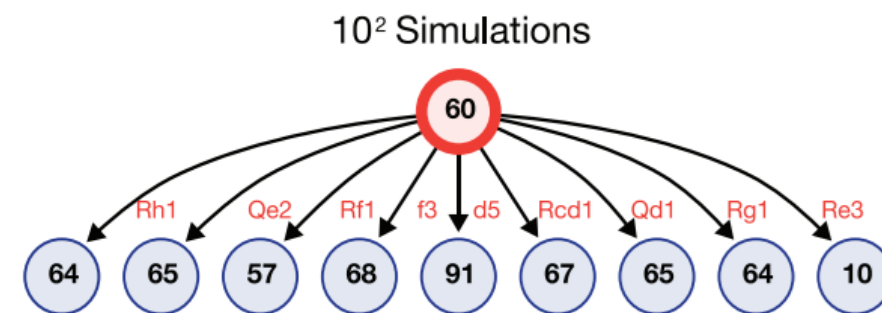
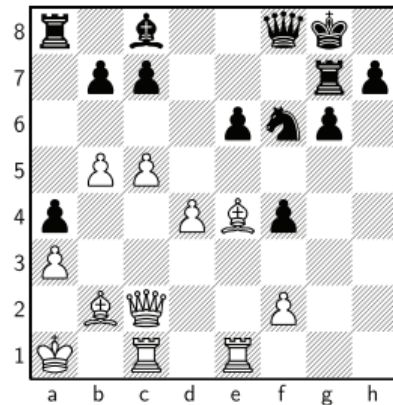
Alphazero

same hyperparameters for all games

train 700,000 steps with mini-batch size 4,096



Alphazero



13.2 explainable AI (XAI)

explaining ML using Shapley values

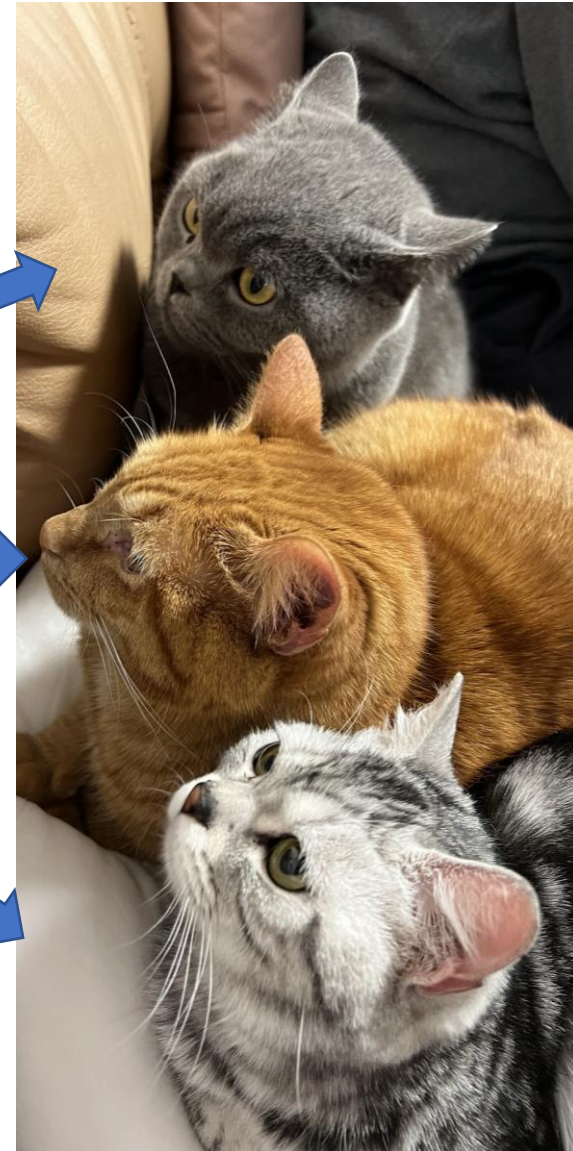
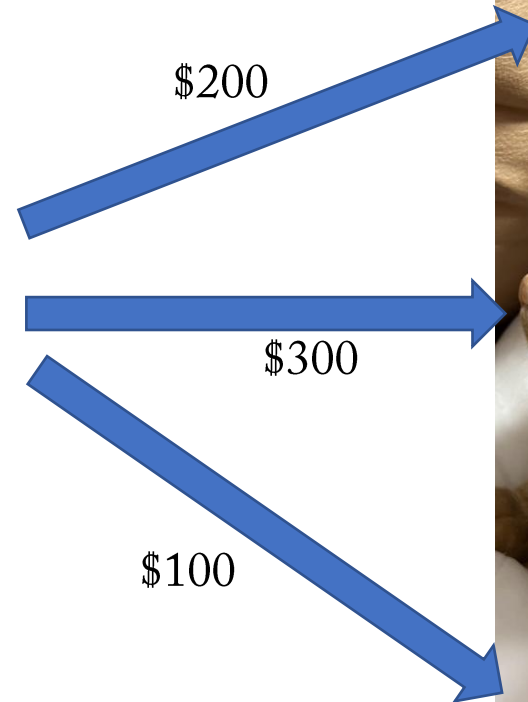
- consider ML as a coalition game with
 - input features $\mathbf{x}$ as the players
 - payoff is the model outcome $f(\mathbf{x})$

the bankruptcy problem

but promised to give the cats



I only have \$100 ...



the proportional rule

but promised to give the cats

I should give $\$100 \cdot 2/6 = 33.33$ ~~\$200~~



I only have \$100 ...

$\$100 \cdot 3/6 = 50$ ~~\$300~~

$\$100 \cdot 1/6 = 16.67$ ~~\$100~~



the constrained equal awards rule

but promised to give the cats

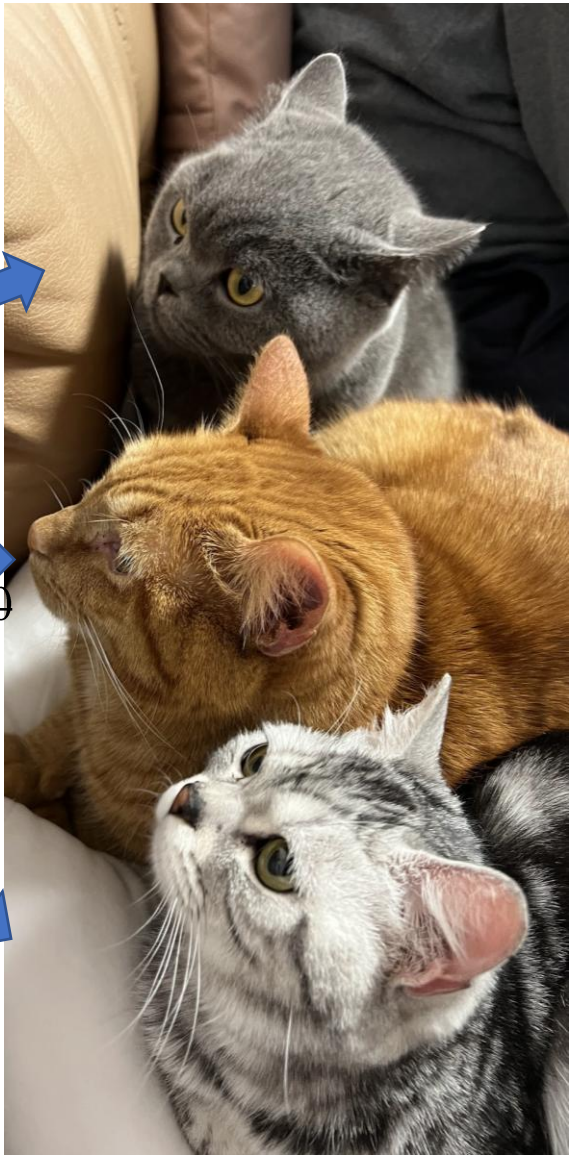
I should give \$33.33 ~~\$200~~



I only have \$100 ...

\$33.33 ~~\$300~~

\$33.33 ~~\$100~~



the contested garment principle

Talmud (Baba Mezi'a 2a, Chapter I, Babylonian Talmud):

Two [persons appearing before a court] hold a garment. If one says, “it is all mine”, and the other says, “half of it is mine”, the former then receives three quarters [of the value of the garment] and the latter receives one quarter

Shapley values

- a cooperative game is played by a set of players $\mathbb{M} = \{1, \dots, M\}$ termed the grand coalition
- the game is characterized by a set function $v : 2^{\mathbb{M}} \rightarrow \mathbb{R}$ such that $v(S)$ is the payoff for any coalition of players S in $\mathbb{M}$, and $v\{\emptyset\} = 0$
- the task is to find fair attributions (give a score) to each player that can be higher or lower than individual payoff depending on the game
- Shapley values attributions are built by averaging the marginal contribution of a player to each possible coalition S

computing Shapley values

- permutation-based approach complexity $O(M!)$

$$\phi_i(v) = \mathbb{E}_{\mathbf{O} \sim \pi(M)} [v(\text{pre}_i(\mathbf{O}) \cup \{i\}) - v(\text{pre}_i(\mathbf{O}))]$$

- subset-based approach complexity $O(2^M)$

$$\phi_i(v) = \frac{1}{M} \sum_{S \subseteq \mathcal{M} \setminus \{i\}} \binom{M-1}{|S|}^{-1} (v(S \cup \{i\}) - v(S))$$

example

$v(\{1\}) = 100$, $v(\{2\}) = 125$, $v(\{3\}) = 50$, $v(\{1,2\}) = 270$, $v(\{1,3\}) = 375$, $v(\{2,3\}) = 350$ and $v(\{1,2,3\}) = 500$

permutation-based approach

Probability	Order of arrival	1's marginal contribution	2's marginal contribution	3's marginal contribution
$\frac{1}{6}$	first 1 then 2 then 3: 123	$v(\{1\}) = 100$	$v(\{1,2\}) - v(\{1\}) = 270 - 100 = 170$	$v(\{1,2,3\}) - v(\{1,2\}) = 500 - 270 = 230$
$\frac{1}{6}$	first 1 then 3 then 2: 132	$v(\{1\}) = 100$	$v(\{1,2,3\}) - v(\{1,3\}) = 500 - 375 = 125$	$v(\{1,3\}) - v(\{1\}) = 375 - 100 = 275$
$\frac{1}{6}$	first 2 then 1 then 3: 213	$v(\{1,2\}) - v(\{2\}) = 270 - 125 = 145$	$v(\{2\}) = 125$	$v(\{1,2,3\}) - v(\{1,2\}) = 500 - 270 = 230$
$\frac{1}{6}$	first 2 then 3 then 1: 231	$v(\{1,2,3\}) - v(\{2,3\}) = 500 - 350 = 150$	$v(\{2\}) = 125$	$v(\{2,3\}) - v(\{2\}) = 350 - 125 = 225$
$\frac{1}{6}$	first 3 then 1 then 2: 312	$v(\{1,3\}) - v(\{3\}) = 375 - 50 = 325$	$v(\{1,2,3\}) - v(\{1,3\}) = 500 - 375 = 125$	$v(\{3\}) = 50$
$\frac{1}{6}$	first 3 then 2 then 1: 321	$v(\{1,2,3\}) - v(\{2,3\}) = 500 - 350 = 150$	$v(\{2,3\}) - v(\{3\}) = 350 - 50 = 300$	$v(\{3\}) = 50$

$$x_1 = 1/6 (100 + 100 + 145 + 150 + 325 + 150) = 970/6$$

example

$v(\{1\}) = 100$, $v(\{2\}) = 125$, $v(\{3\}) = 50$, $v(\{1,2\}) = 270$, $v(\{1,3\}) = 375$, $v(\{2,3\}) = 350$ and $v(\{1,2,3\}) = 500$

subset-based approach

$$M = 3, \quad i = 1;$$

$\mathbb{S}$: subsets of $M \setminus \{1\} = \{2,3\}, \{2\}, \{3\}, \emptyset$;

$$\binom{M-1}{|\mathbb{S}|} : \binom{2}{2} = 1, \binom{2}{1} = 2, \binom{2}{0} = 1$$

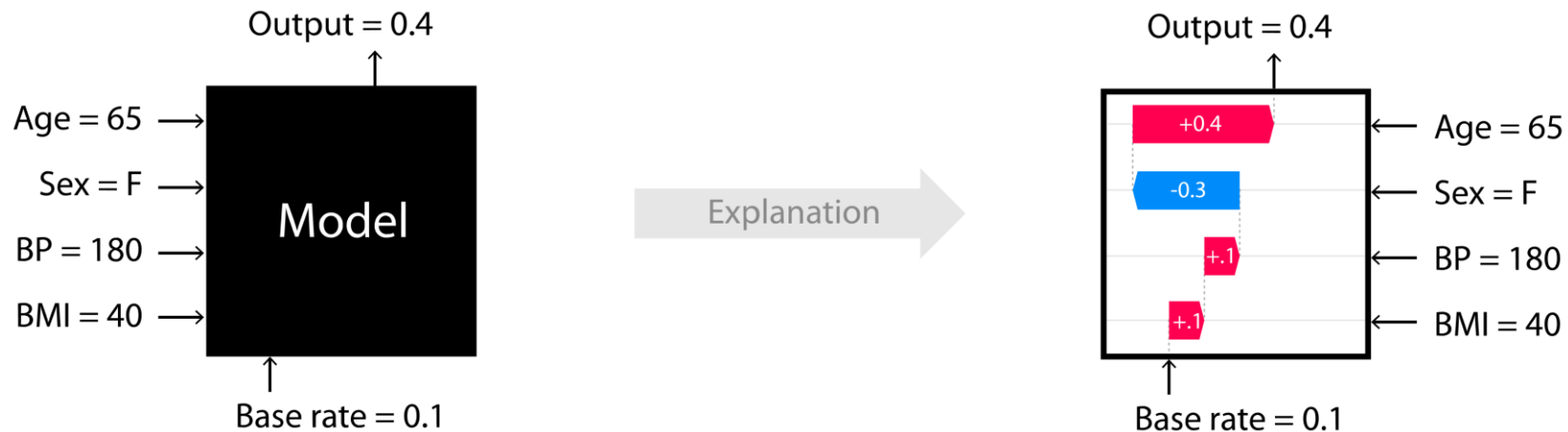
$$x_1 = \frac{1}{3} \left(1(v(1,2,3) - v(2,3)) + \frac{1}{2}(v(1,2) - v(2)) + \frac{1}{2}(v(1,3) - v(3)) + 1(v\{1\} - v\{\emptyset\}) \right)$$

$$= \frac{1}{3} (150 + 0.5 \times 145 + 0.5 \times 325 + 100) = 970/6$$

SHAP = SHapley Additive exPlanations



SHAP



SHAP

- debugging model predictions
- generating an explanation for the end-user
- analyzing model robustness
- extracting rules from the model

SHAP

- Shapley values tell us how to fairly distribute the “payout” (= the prediction) among the features
- a player can be an individual feature value, e.g., for tabular data; a player can also be a group of feature values, e.g., to explain an image, pixels can be grouped to superpixels and the prediction distributed among them
- one innovation of SHAP is that the Shapley value explanation is represented as a linear model

SHAP

- SHAP specifies the explanation as:

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j$$

coalition vector

maximum coalition size

coalition value

the feature attribution for a feature j , the Shapley values

in the coalition vector, an entry of 1 means that the corresponding feature value is “present” and 0 that it is “absent”

SHAP

- for $\mathbf{x}$, the instance of interest, the coalition vector $\mathbf{x}'$ is a vector of all 1's, i.e. all feature values are “present”
- the formula simplifies to:

$$g(\mathbf{x}') = \phi_0 + \sum_{j=1}^M \phi_j$$

SHAP

- we need to have a regime for defining ϕ_j , which satisfies:

1. local accuracy

$$\hat{f}(x) = g(x') = \phi_0 + \sum_{j=1}^M \phi_j x'_j$$

2. missingness

$$x'_j = 0 \Rightarrow \phi_j = 0$$

3. consistency

$$\hat{f}'_x(z') - \hat{f}'_x(z'_{-j}) \geq \hat{f}_x(z') - \hat{f}_x(z'_{-j}) \Rightarrow \phi_j(\hat{f}', x) \geq \phi_j(\hat{f}, x)$$

SHAP

- only one model satisfies the above conditions:

$$\phi_i(f, x) = \sum_{z' \subseteq x'} \frac{|z'|!(M - |z'| - 1)!}{M!} [f_x(z') - f_x(z' \setminus i)]$$

where $|z'|$ is the number of non-zero entries in z' , and $z' \subseteq x'$ represents all z' vectors where the non-zero entries are a subset of the non-zero entries in x' .

kernel SHAP

1. sample coalitions $z'_k \in \{0,1\}^M, k = \{1, \dots, K\}$
2. get prediction for each z'_k by first converting z'_k to the feature space and then applying the model $\hat{f}$
3. compute the weight for each z'_k with the SHAP kernel
4. fit weighted linear model
5. return Shapley values ϕ_k

converting z'_k to the feature space

Coalitions $\xrightarrow{h_x(z')}$ Feature values

Instance x

maps to the feature vector

Age	Weight	Color
1	1	1

Age	Weight	Color
0.5	20	Blue

Instance with
"absent"
features

maps to the values of a randomly sampled data instance

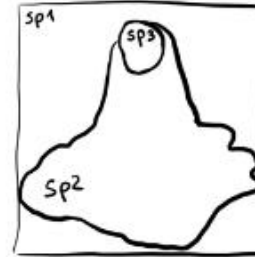
Age	Weight	Color
1	0	0

Age	Weight	Color
0.5	20	Blue
	↓	↓
	17	Pink

converting z'_k to the feature space

Coalitions of
super pixels $\xrightarrow{h_x(z')}$ Image

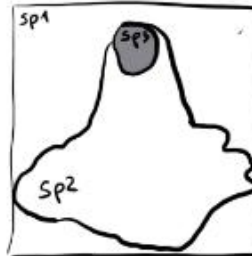
Instance x



sp1	sp2	sp3
1	1	1



Instance x
with absent
features



sp1	sp2	sp3
1	1	0



the SHAP kernel

- Lundberg et al. propose the SHAP kernel:

$$\pi_x(z') = \frac{(M-1)}{\binom{M}{|z'|} |z'| (M - |z'|)}$$

- recall $g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j$

- we optimize

$$L(\hat{f}, g, \pi_x) = \sum_{z' \in Z} [\hat{f}(h_x(z')) - g(z')]^2 \pi_x(z')$$

Thank you!

Reference

- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.
- <https://www.davidsilver.uk/teaching/>
- <https://deepmind.com/learning-resources/reinforcement-learning-series-2021>
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ... & Hassabis, D. (2017). Mastering the game of go without human knowledge. *nature*, 550(7676), 354-359.
- <https://christophm.github.io/interpretable-ml-book/shap.html>

